

In 2013 I tried working with FPGA.

see

https://github.com/develone/zedboard_yocto/blob/master/doc/Zedboard_Yocto_test.pdf

see

<https://github.com/develone/07118catzip/blob/master/doc/zipcpu.pdf>

see

<https://github.com/develone/102121icozip/blob/catzip/doc/Working-Software-092821.pdf>

This was the starting point with the original lifting step code provided by ZipCPU

git clone <https://github.com/develone/zipcpu-dwt-lifting.git>

```
devel@pi4-50:~/zipcpu-dwt-lifting/src $ less ~/.octaverc
graphics_toolkit("gnuplot");
```

The .octaverc is required for octave.

```
devel@pi4-50:~/zipcpu-dwt-lifting/src $ ./buildpi_lift.sh
```

```
devel@pi4-50:~/zipcpu-dwt-lifting/host $ ./pi_jpeg 0 1
```

```
devel@pi4-50:~/zipcpu-dwt-lifting/host $ octave
```

GNU Octave, version 7.3.0

Copyright (C) 1993-2022 The Octave Project Developers.

This is free software; see the source code for copying conditions.

There is ABSOLUTELY NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. For details, type 'warranty'.

Octave was configured for "aarch64-unknown-linux-gnu".

Additional information about Octave is available at <https://www.octave.org>.

Please contribute if you find this software useful.

For more information, visit <https://www.octave.org/get-involved.html>

Read <https://www.octave.org/bugs.html> to learn how to submit bug reports.

For information about changes from previous versions, type 'news'.

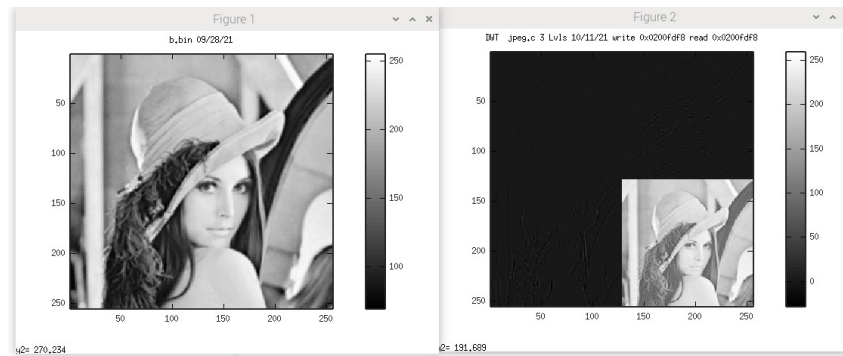
warning: using the gnuplot graphics toolkit is discouraged

The gnuplot graphics toolkit is not actively maintained and has a number of limitations that are unlikely to be fixed. Communication with gnuplot uses a one-directional pipe and limited information is passed back to the Octave interpreter so most changes made interactively in the plot window

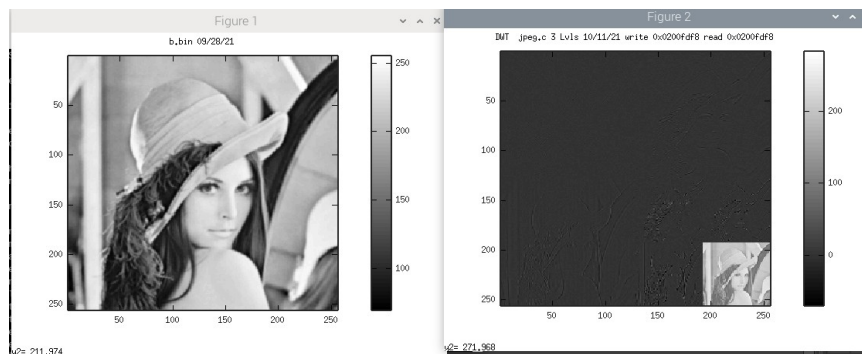
will not be reflected in the graphics properties managed by Octave. For example, if the plot window is closed with a mouse click, Octave will not be notified and will not update its internal list of open figure windows. The qt toolkit is recommended instead.

```
octave:1>
```

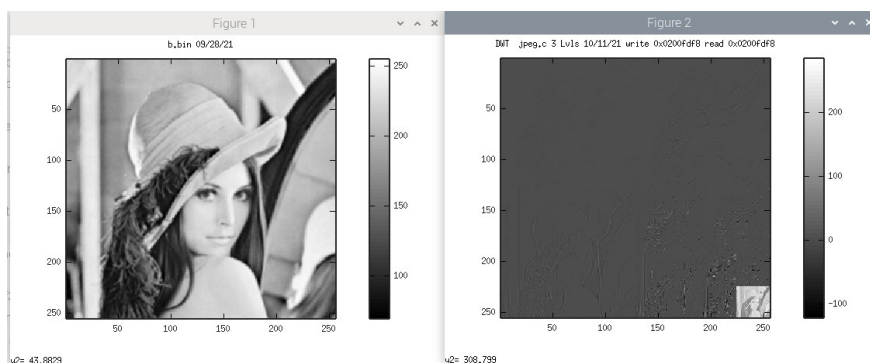
```
octave:1> rgb
```



1 lvl 128x128



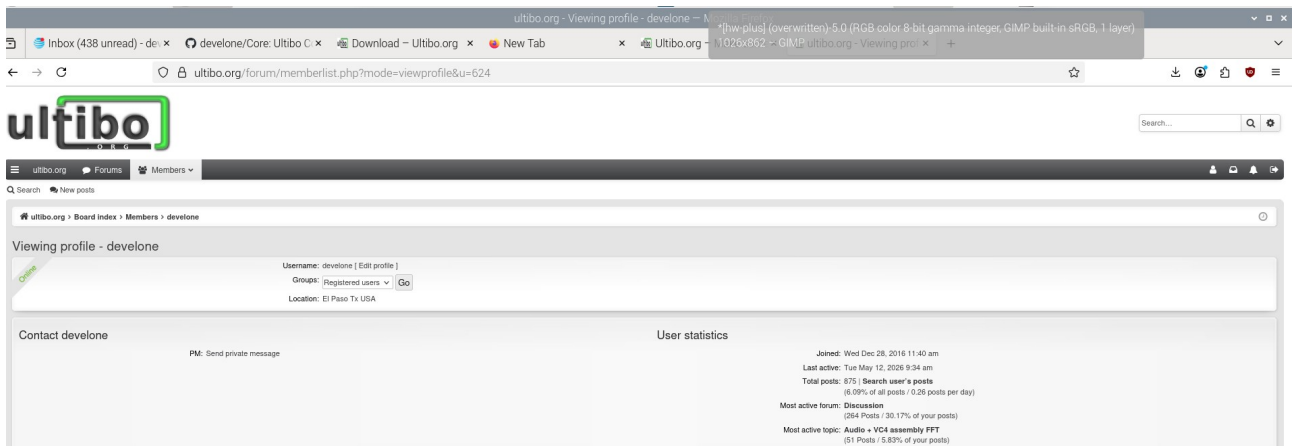
2 lvlis 64x64



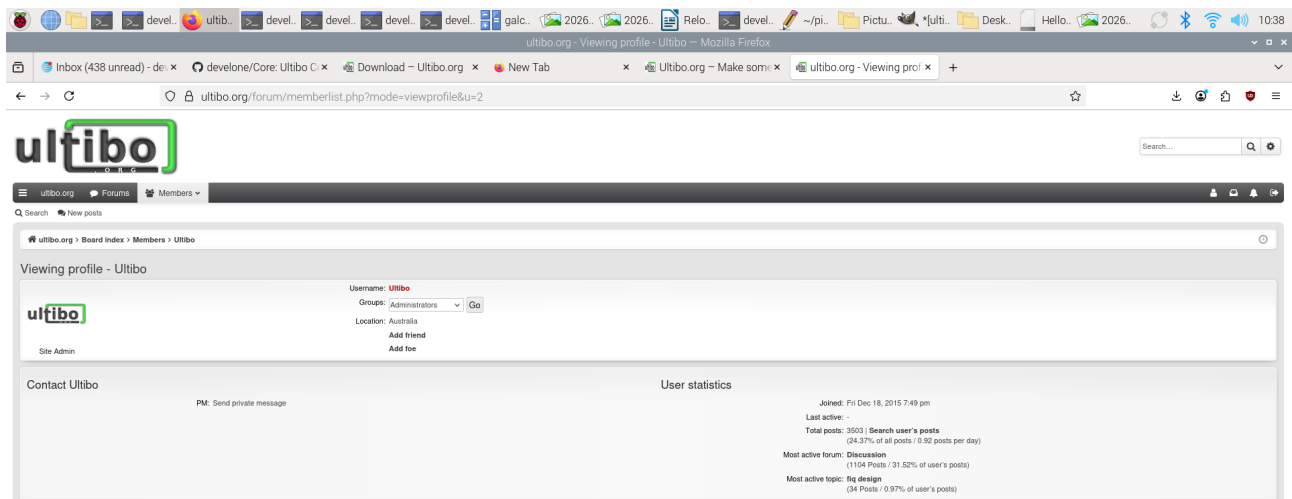
3 lvlis 32x32

I started working with Ultibo since 2016.

X



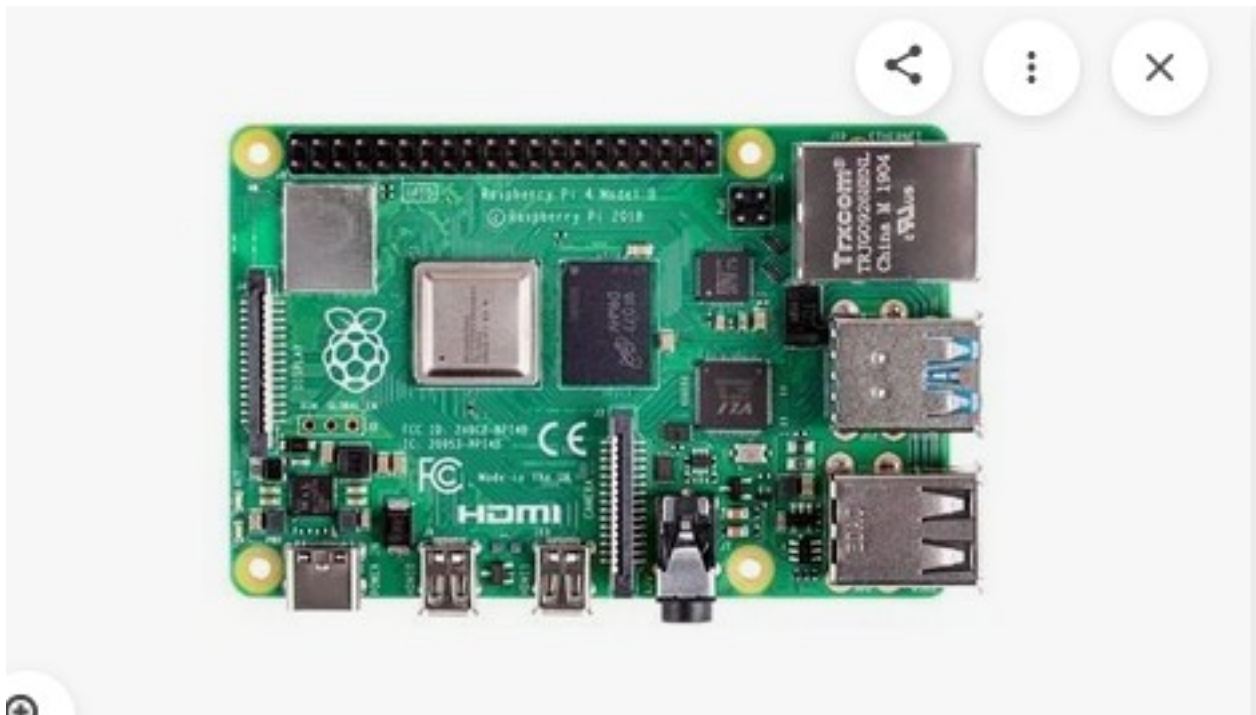
This was shortly after the projected started in 2015,



X

2025-11-24-raspbios-bookworm-arm64.img

Installed bookworm release on Raspberry Pi4 4Gb pi4-50



x

```
devel@pi4-50:~ $ shasum extra64.sh
1d92b567c492321c21ce7baaeebfd7e0cf83db3d  extra64.sh
```

```
curl https://pyenv.run | bash
```

05/12/26

```
devel@pi5-90:/media/devel/1b763776-4e1d-499c-9f24-a116a58c161f $ ls 64bit-pi4-26/
```

```
extra_pkgs.sh  extra_pkgs.sh.orig
```

```
installed-openocd082722-228ede-64bit.img
```

```
openocd082722-228ede-64bit.img
```

```
qemu-6.2.0-64bit.img  qemu-6.2.0-rpios-64bit.img
```

```
devel@pi4-50:~ $ sudo unsquashfs -d qemu-6.2.0 qemu-6.2.0-64bit.img
```

```
devel@pi4-50:~ $ sudo unsquashfs -d qemu-6.2.0-rpios qemu-6.2.0-rpios-64bit.img
```

```
qemu.sh
```

```
#!/bin/bash
```

```
export QEMU_PATH=$HOME/qemu-6.2.0-rpios/bin
```

```
export PATH=$QEMU_PATH:$PATH
```

```
echo $PATH
```

```
devel@pi4-50:~ $ ./ultiboinstaller.sh
```

Linux installer for Free Pascal and Lazarus (Ultibo edition)

This installation will download the sources for:

- Ultibo core

- Ultibo tools

- Ultibo examples

- Free Pascal (Ultibo edition)

- Lazarus IDE (Ultibo edition)

Then it will build all of the above, this will take several minutes to complete depending on the speed of your system.

The installation will not interfere with any existing development environments including other installations of Free Pascal and Lazarus.

Continue (y/n)?

Do you want to build and install the Lazarus IDE (y/n)?

These can be installed on Debian based distributions using:

```
sudo apt-get install build-essential gdb-minimal unzip
```

Lazarus IDE requires the GTK2 and X11 dev packages which can be installed on Debian based distributions by using:

```
sudo apt-get install libgtk2.0-dev libcairo2-dev \
  libpango1.0-dev libgdk-pixbuf2.0-dev libatk1.0-dev \
  libghc-x11-dev
```

(On some distributions libgdk-pixbuf2.0-dev might have been replaced by libgdk-pixbuf-xlib-2.0-dev)

Cross compiling Ultibo applications from Linux requires the arm-none-eabi and aarch64-linux-gnu builds of the binutils package, these can be installed on Debian based distributions using:

```
sudo apt-get install binutils-arm-none-eabi \
  binutils-aarch64-linux-gnu
```

Press return to check for these prerequisites

Enter an installation folder or press return to accept the default install location

[/home/devel/ultibo/core]:

The install folder will be:

/home/devel/ultibo/core

Continue? (y,n):

After install do you want a shortcut created in:

/home/devel/.local/share/applications (y/n)?

Do you want to build the Hello World examples (y/n)?

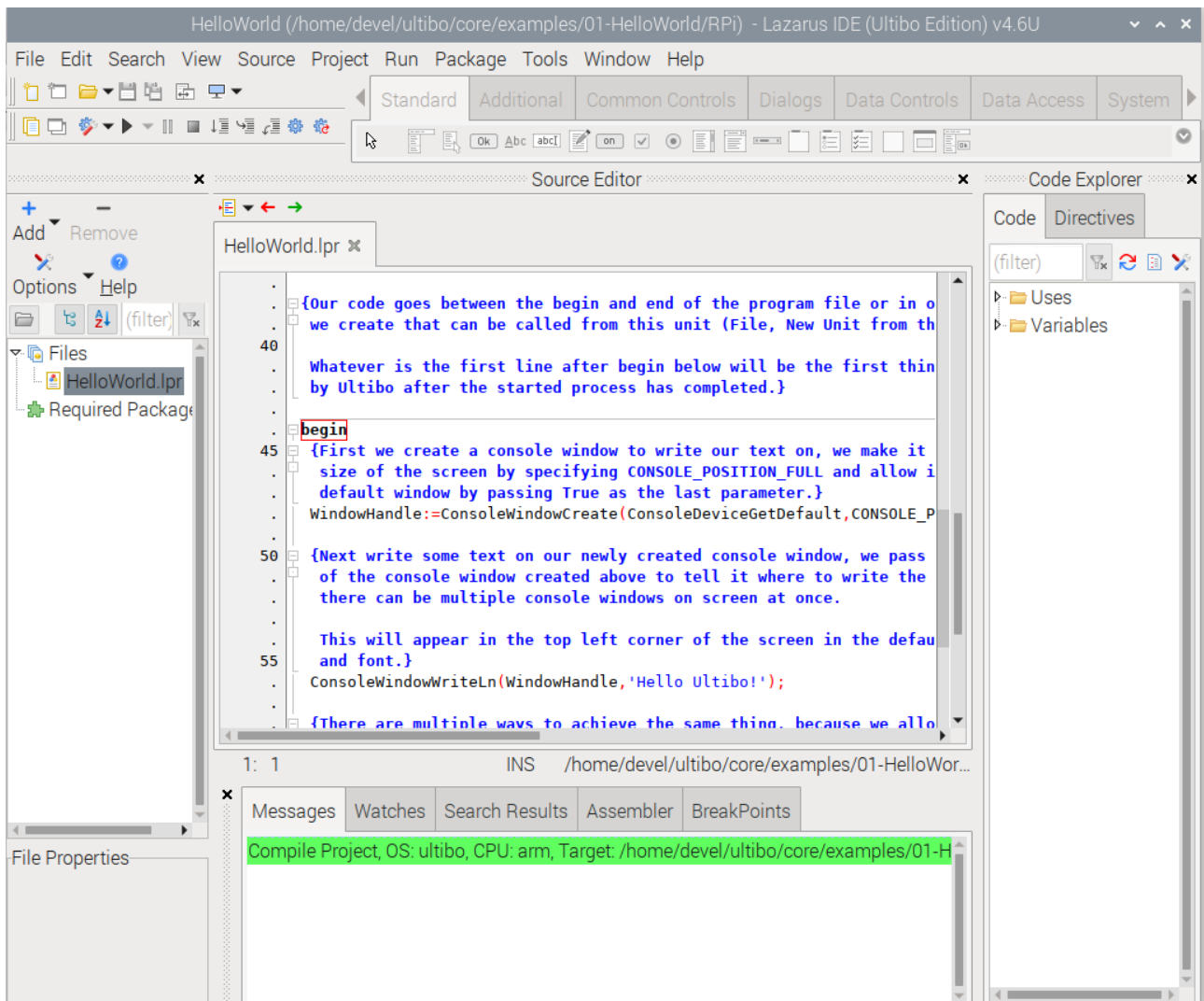
Tested Raspbery Pi-Zero

x



x

This is an example of the HelloWorld for the Raspberry Pi-Zero.



Green bar indicates that the program compiled ok.

X

X

devel@pi4-50:~/Ultibo_Projects/HelloWorld-plus/RPi \$ diff HelloWorld.lpr

~/ultibo/core/examples/01-HelloWorld/RPi/HelloWorld.lpr

57,62c57

< ConsoleWindowWriteLn(WindowHandle,'This program was compiled using');

< ConsoleWindowWriteLn(WindowHandle,'on a Raspberry Pi4 4GB');

< ConsoleWindowWriteLn(WindowHandle,'running Lazarus IDE Ultibo Edition');

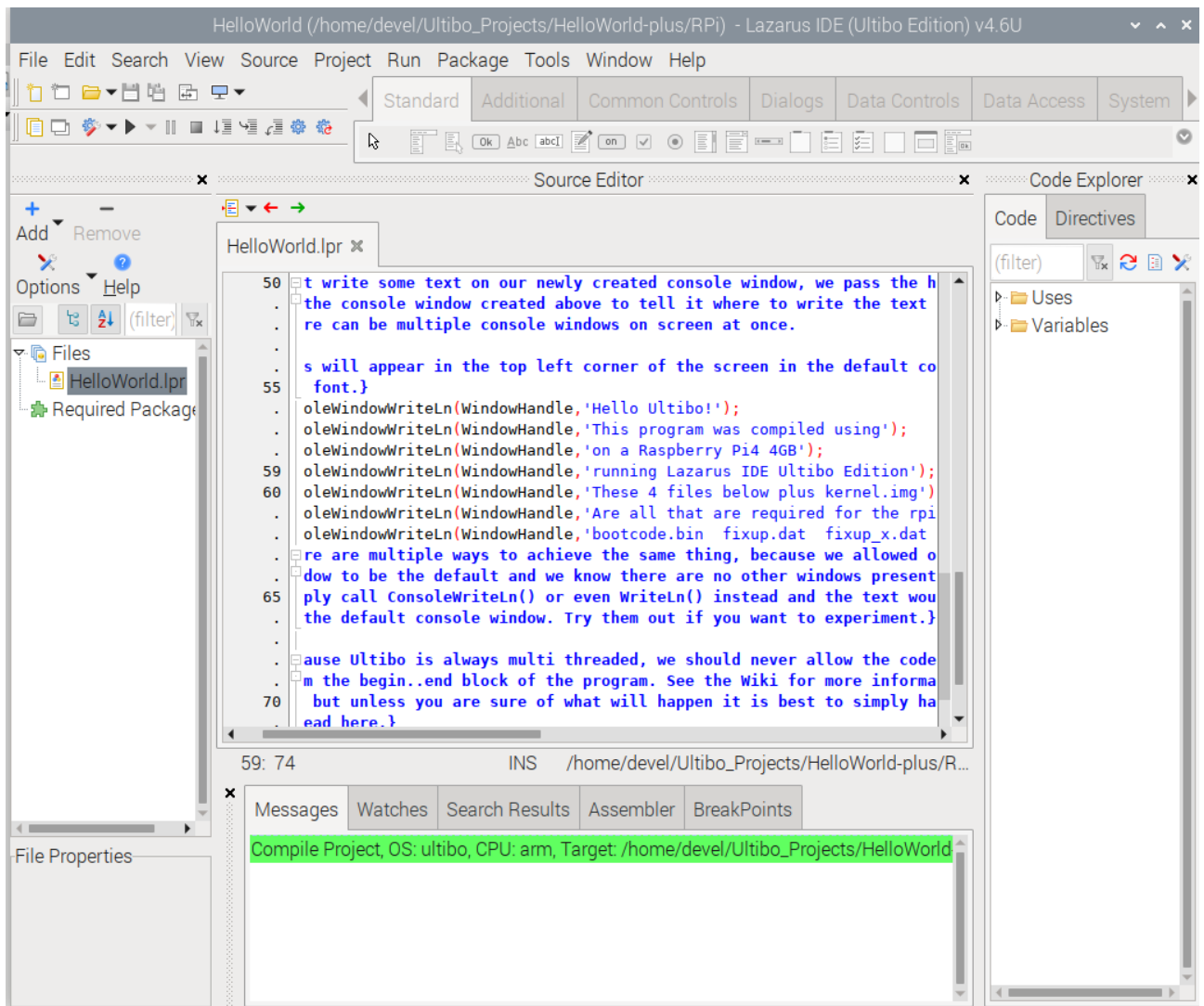
< ConsoleWindowWriteLn(WindowHandle,'These 4 files below plus kernel.img');

< ConsoleWindowWriteLn(WindowHandle,'Are all that are required for the rpi-zero');

< ConsoleWindowWriteLn(WindowHandle,'bootcode.bin fixup.dat fixup_x.dat start4x.elf
start.elf');

>

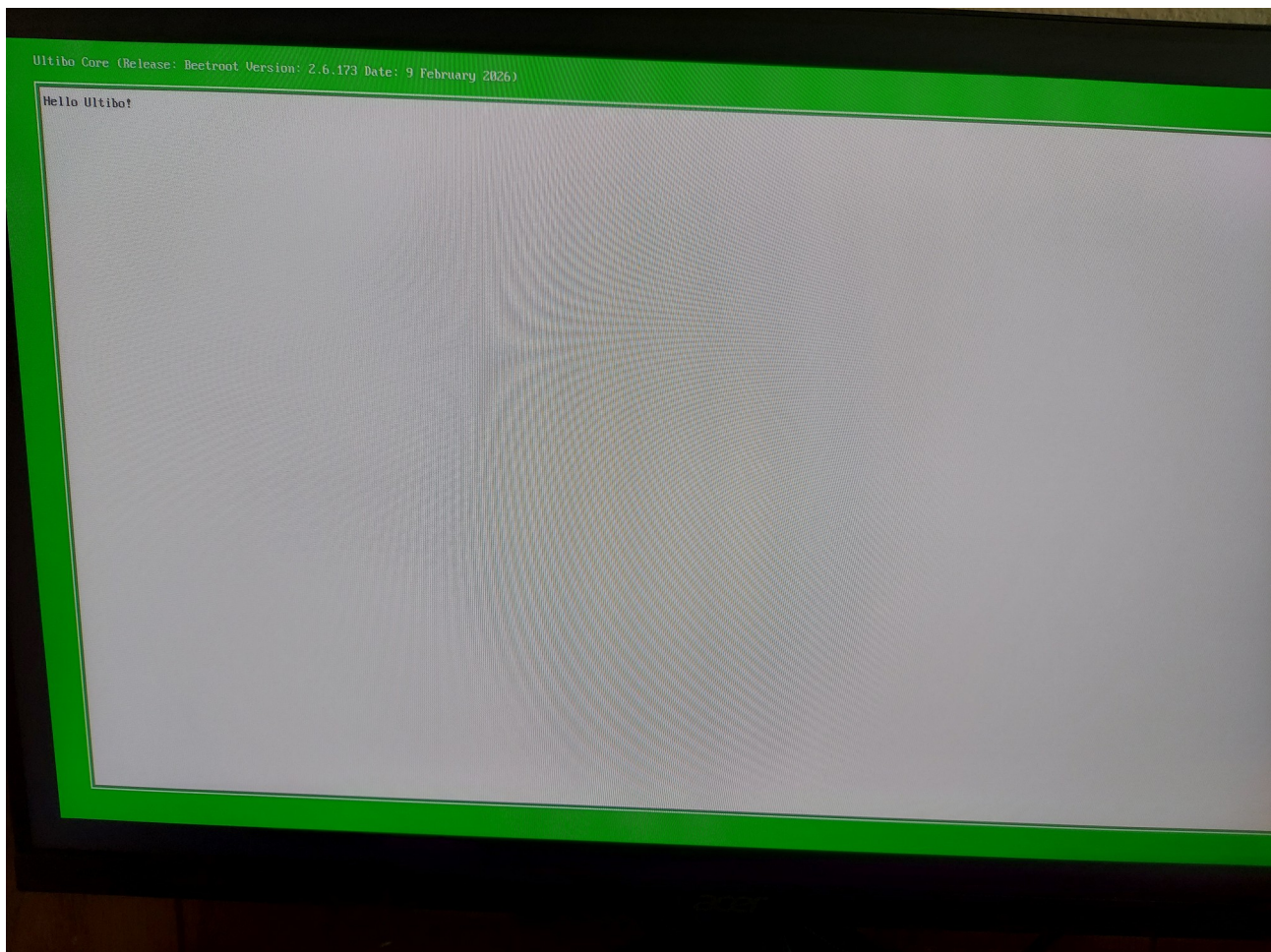
Green bar indicates that the program compiled ok.



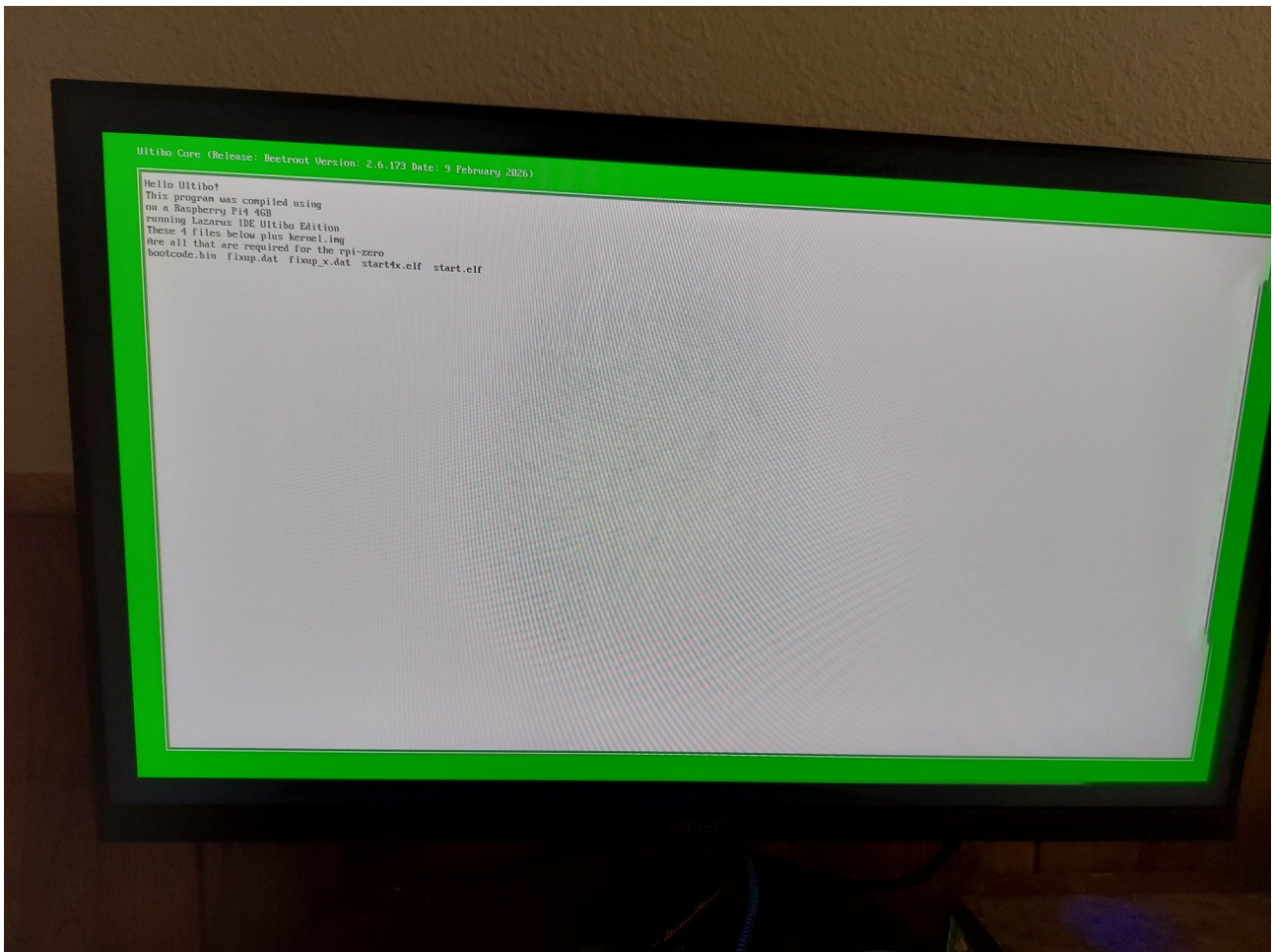
```

devel@pi4-50:~/ultibo/core/examples/01-HelloWorld/RPi $ ls ~/firmwar_for_ultibo/112221/
bootcode.bin fixup.dat fixup_x.dat start4x.elf start.elf plus kernel.img

```

```
devel@pi4-50:~/ultibo/core/examples/01-HelloWorld/RPi $ ls ~/firmwar_for_ultibo/112221/  
bootcode.bin  fixup.dat  fixup_x.dat  start4x.elf  start.elf  plus  kernel.img
```



The next phase of testing was with Ultibo and the Raspberry Pi2.



Raspberry Pi 2 Model B Desktop (Quad Core CPU 900 MHz, 1 GB RAM, Linux)

[Visit the Raspberry Pi Store](#)

4.7 ★★★★★ (4,593)

Amazon's Choice

\$64⁹⁹

Get \$10 off instantly: Pay **\$54.99** ~~\$64.99~~ upon approval for the Amazon Store Card. No annual fee.

Style: **Desktop**

Brand	Raspberry Pi
Model Name	Raspberry Pi 2 Model B
Ram Memory Installed	1 GB
Size	
CPU Model	Cortex A7

\$64⁹⁹

FREE delivery **May 15 - 19.**

[Details](#)

Or fastest delivery **Thursday, May 14. Order within 14 mins.**

[Details](#)

📍 Delivering to El Paso 79936 - [Update location](#)

In Stock

Quantity: 1

[Add to cart](#)

[Buy Now](#)

X

```
devel@pi4-50:~/Ultibo_Projects/jpeg2000/src $ ./compile_ultibo.sh
```

```
devel@pi4-50:~/Ultibo_Projects/jpeg2000/src $ ls *.c | wc
  22   22  182
```

```
devel@pi4-50:~/Ultibo_Projects/jpeg2000/src $ ls *.c
bio.c      invert.c  opj_clock.c  t2.c
cio.c      j2k.c     opj_malloc.c  tcd.c
dwt.c      jp2.c     pi.c          tgt.c
event.c    mct.c     rd-wr-ops.c  thread.c
```

function_list.c mqc.c sparse_array.c
image.c openjpeg.c t1.c

devel@pi4-50:~/Ultibo_Projects/jpeg2000/QEMU \$./libbuild.sh

dwtlift.c: In function 'decompress':

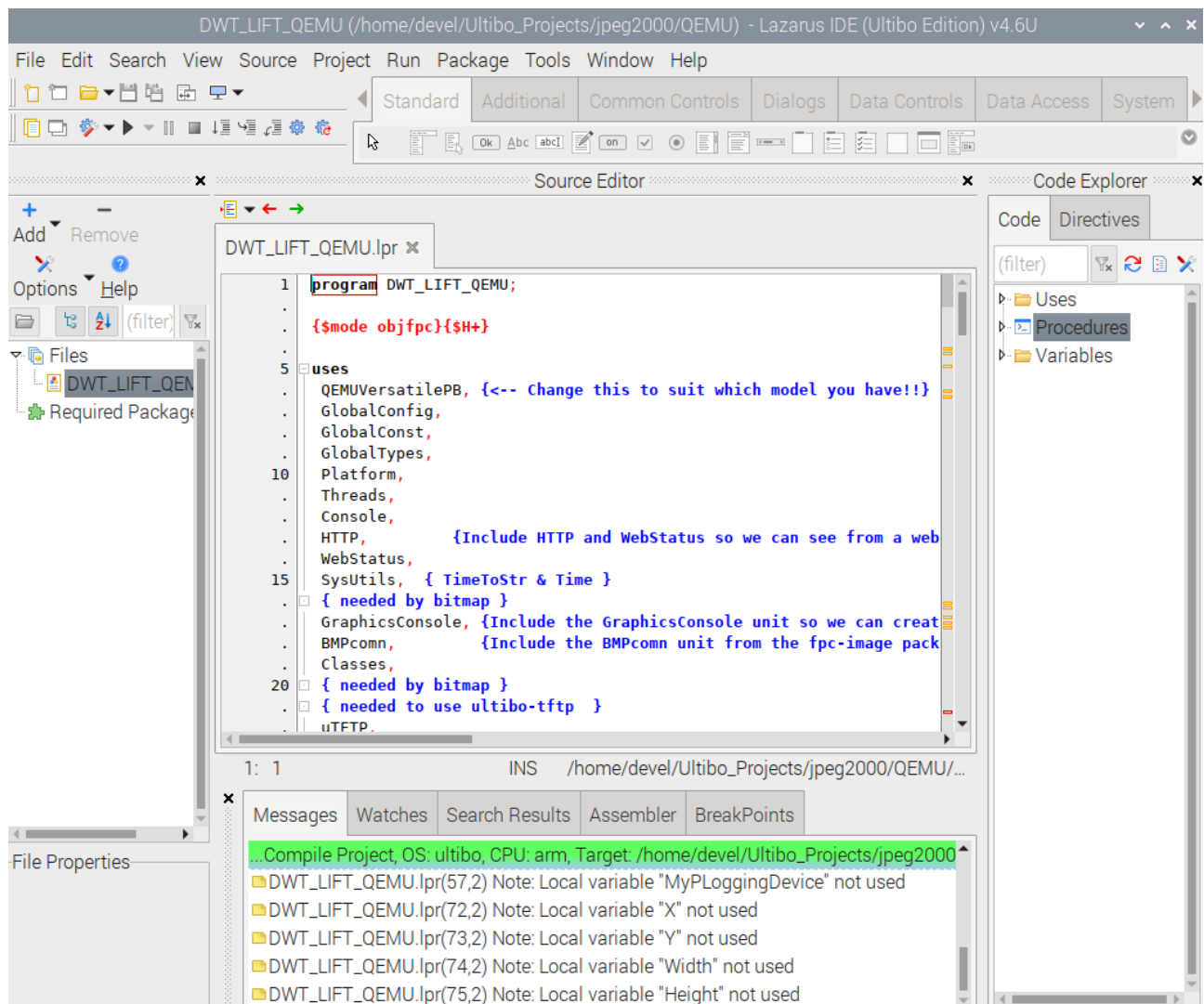
dwtlift.c:658:17: warning: implicit declaration of function 'octave_write_byte' [-Wimplicit-function-declaration]

```
658 |         octave_write_byte(r_decompress_fn,r_decompress,da_x1*da_y1);
    |         ^~~~~~
```

when ./libbuild.sh is executed should be 23

23 23 192 libdwtlift_obj.txt

X



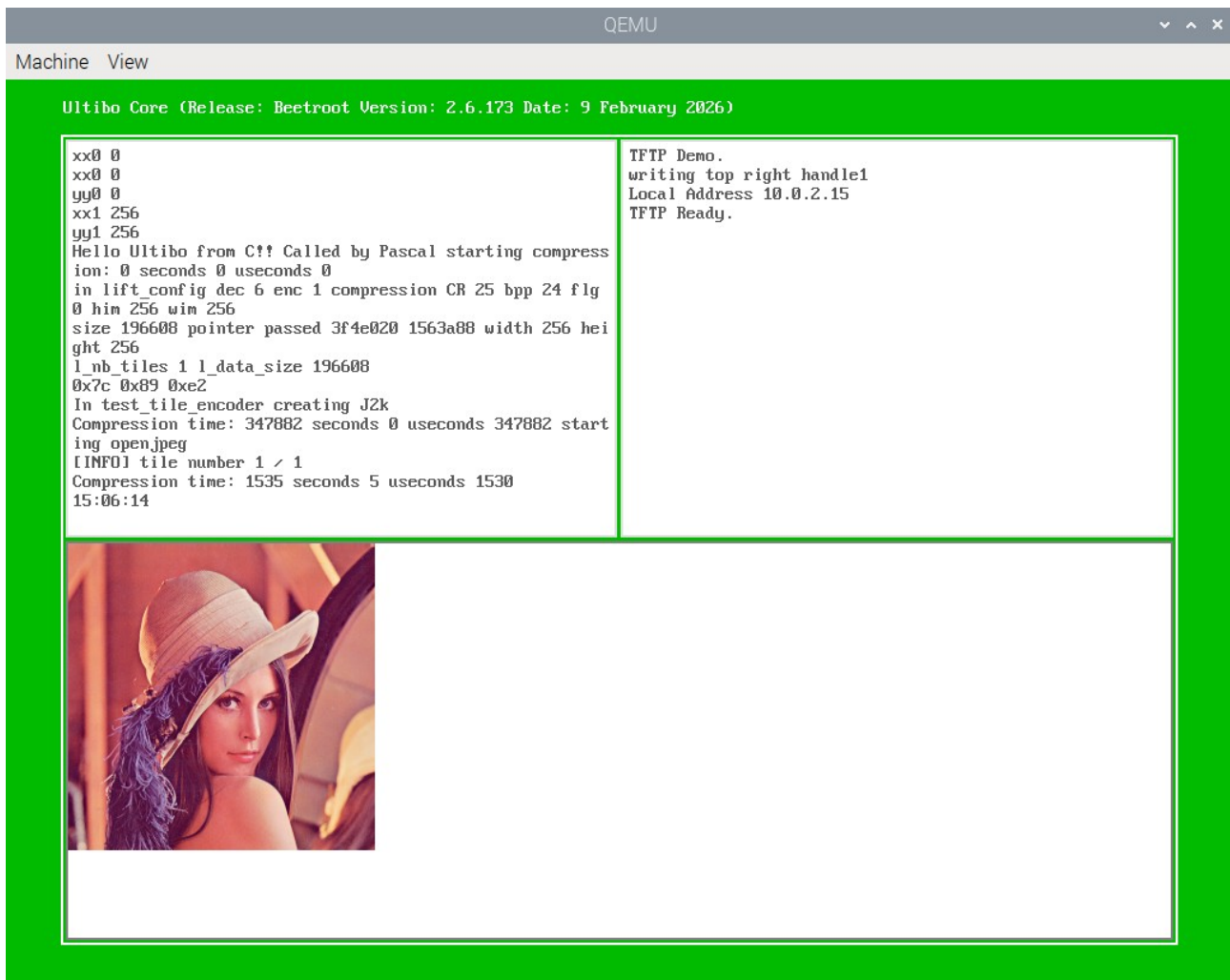
X

devel@pi4-50:~/Ultibo_Projects/jpeg2000/QEMU \$. ~/qemu.sh

/home/devel/qemu-6.2.0-rpios/bin:/home/devel/.pyenv/plugins/pyenv-virtualenv/shims:/home/devel/.pyenv/shims:/home/devel/.pyenv/bin:/home/devel/local/openocd/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games

devel@pi4-50:~/Ultibo_Projects/jpeg2000/QEMU \$./startqemu.sh

X



The image above indicates that kernel7.img.jpeg is the file kernel7.img

devel@pi5-70:~ \$ telnet 192.168.12.161

C:\> dir

Ultibo Core (Release: Beetroot Version: 2.6.173 Date: 9 February 2026)
(Type HELP for a list of available commands)

Directory of C:\

2026-05-11 22:42:30	24 config-1.txt
2026-05-09 16:12:42	52460 bootcode.bin
2026-05-09 16:12:42	7221 fixup.dat
2026-05-09 16:12:42	10197 fixup_x.dat
2026-05-09 16:12:42	2963616 start.elf
2026-05-09 16:12:42	2991400 start4x.elf
2026-05-09 16:10:36	24 256com
2026-05-09 16:10:36	24 256decom
2026-05-09 16:10:36	196730 lena_rgb_256.bmp
2026-05-11 18:33:08	24 testfile
2026-05-12 21:19:09	7848 test.j2k
2026-05-11 18:32:30	196730 MyBitmap.bmp
2026-05-11 18:19:28	3787872 kernel7.img
2026-05-12 01:36:28	2996612 kernel7.img.dwt

```
2026-05-11 18:19:28      3787872 kernel7.img.jpeg
2026-05-12 01:39:28      196662 MySavedBitmap.bmp
16 file(s) 17195316 bytes
0 dir(s)
```

```
C:\> del kernel7.img
```

```
C:\> copy kernel7.img.dwt kernel7.img
```

```
1 file(s) copied
```

```
C:\> dir
```

```
Directory of C:\
```

```
2026-05-11 22:42:30      24 config-1.txt
2026-05-09 16:12:42     52460 bootcode.bin
2026-05-09 16:12:42      7221 fixup.dat
2026-05-09 16:12:42     10197 fixup_x.dat
2026-05-09 16:12:42    2963616 start.elf
2026-05-09 16:12:42    2991400 start4x.elf
2026-05-09 16:10:36      24 256com
2026-05-09 16:10:36      24 256decom
2026-05-09 16:10:36    196730 lena_rgb_256.bmp
2026-05-11 18:33:08      24 testfile
2026-05-12 21:19:09      7848 test.j2k
2026-05-11 18:32:30    196730 MyBitmap.bmp
2026-05-12 01:36:28    2996612 kernel7.img.dwt
2026-05-11 18:19:28    3787872 kernel7.img.jpeg
2026-05-12 01:39:28      196662 MySavedBitmap.bmp
2026-05-12 01:36:28    2996612 kernel7.img
16 file(s) 16404056 bytes
0 dir(s)
```

The list above indicates that kernel7.img.dwt is the file kernel7.img

```
C:\> restart
```

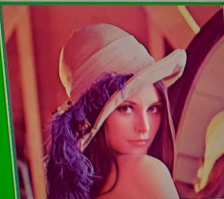
Restarting in 1000 milliseconds

The image below will now be displayed.

bo Core (Release: Beetroot Version: 2.6.173 Date: 9 February 2026)

```
xyz
p 24
ze 196608
inter passed f20020
. 65536.000000 sqrt tt 256.000000 256 256
scal char ptr f20020
llocating memory with malloc
mg->n_red 0xc00004f0
mg->n_green 0xc00004f0
mg->n_blue 0xc00004f0
mg->n_tnp 0xc00004f0
copying RGB 8 bit char to 32 int
mg->n_red 0xc00004f0
mg->n_green 0xc00004f0
mg->n_blue 0xc00004f0
reseting pointers
mg->n_red 0xc00004f0 passed ptr 0xf20020
mg->n_green 0xc00004f0
mg->n_blue 0xc00004f0
Calling lifting red
Calling lifting green
Calling lifting blue
lifting to Buffer
Elapsed time: 34412 microseconds
156
01:39:29
```

```
writing top right handle1
Buffer 00f20020 Size 196608 LineSize 768 BitCount 24
UPSIDEDOWN
Going to free Buffer memory
Local Address 192.168.12.161
TFTP Ready.
```



Bitmap file saved successfully

